

MA305, Fall 2025

Embry-Riddle Aeronautical University

Numerical Methods for Ordinary Differential Equations

Wylie Roberts, Hillary Spang, and Clancy Crawford

December 8, 2025

Abstract

Contents

1	Introduction	3
2	Problem Statement	3
2.1	Part I	3
2.2	Part II	3
2.3	Part III	4
3	Method/Analysis	5
3.1	Part I	5
3.2	Part II	6
3.3	Part III	7
4	Solutions/Results	7
4.1	Part I	7
4.2	Part II	10
4.3	Part III	11
5	Discussion/Conclusions	12
5.1	Part I	12
5.2	Part II	13
5.3	Part III	13
A	Python Codes	14

1 Introduction

Ordinary differential equations are used to model physical, dynamic systems with respect to a single variable. In some cases, the exact solutions to these equations are not easily determined. Numerical methods are utilized to provide accurate approximations, while requiring significantly less computational power. By using discrete points of the function, the area beneath the curve can be approximated by rectangles, or more accurately, trapezoids. These approximations vary widely in accuracy, but often provide excellent results when compared to exact answers.

2 Problem Statement

2.1 Part I

Part I of the project focused on approximating a definite integral through 4 different methods and comparing their final results to the exact answer. The definite integral in question is

$$\int_0^1 e^{-x^2} dx$$

The four methods utilized are the Midpoint Rule, the Trapezoid Rule, Simpson's Rule, and the Two-Point Gauss Quadrature rule. The objective of the experiment is to compare the accuracy of each method using the percent difference with respect to a known definite integral.

Given:

$$\int_0^1 e^{-x^2} dx = 0.74682413$$

2.2 Part II

The goal of this part is to approximate the area of an irregularly shaped plot of land using the measured width values provided at evenly spaced positions along the x -axis. Once the area is computed, it is used to determine the volume of mulch needed to cover the region with a uniform 3-inch layer, and then the number of 3 ft³ mulch bags required.

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
y	8	10	11	15	16	16	16	16	15.5	15.5	15.5	15	15	15	14.5	14.5	14

x	17	18	19	20	21	22	23	24	25	26
y	14	14	13.5	13	13	13	12	11	19	6

Figure 1: Width measurements (in feet) of the irregularly shaped land.

The width measurements of the land at integer values of x from 0 to 26 are shown in Figure 1. These measurements represent the function values $y = f(x)$ sampled at equal intervals of 1 ft, making them suitable for numerical integration. To approximate the area, the code applies the Composite Simpson's Rule. This method gives an accurate estimate of the definite integral when the data points are evenly spaced. The formula is:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{i=1,3,5,\dots}^{N-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{N-2} f(x_i) + f(x_N) \right],$$

$$h = \frac{b-a}{N}, x_i = a + ih.$$

2.3 Part III

Part 3 of this problem uses Euler's Method to Estimate The World's Population from 1950 to 2020. It uses a logistical growth differential equation to approximate the solution.

Given Data:

- $p_0(in1950) = 2555millionpeople$
- $k_{rm} = 0.026/yr$
- $p_{max} = 12000$ million people
- t vs p graph for comparison:

Year	Population (millions)
1950	2555
1960	3040
1970	3708
1980	4454
1990	5276
2000	6079
2010	6922
2020	7753

Table 1: Given world population data from 1950–2020.

The differential equation is as follows:

$$\frac{dp}{dt} = k_{rm} \left(1 - \frac{p}{p_{\max}} \right) p$$

The Euler's Method is as follows:

$$p_{n+1} = p_n + h f(p_n)$$

3 Method/Analysis

3.1 Part I

The main objective is to develop algorithms for each of the four methods in order to compare them with a known value. The first method, the Midpoint Rule can be shown as

$$\int_a^b f(x)dx \approx h \sum_{i=1}^N f(x_i), h = \frac{b-a}{n}, x_i = a + \frac{2i-1}{2}h$$

The advantage to the Midpoint Rule is its overall simplicity. By finding the average between two points and producing a rectangular approximation of the area, it provides a fairly rough approximation. but as the step size decreases, and the endpoints approach the midpoint on either side, the approximation becomes more accurate. The trapezoid rule can be shown as:

$$\int_a^b f(x)dx \approx h[\frac{1}{2}f(x_0) + \sum_{i=1}^{N-1} f(x_i) + \frac{1}{2}f(x_n)], h = \frac{b-a}{n}, x_i = a + ih$$

The trapezoid rule differs from the midpoint rule by defining a trapezoid with the endpoints (a and b) and the x-axis. This provides a better approximation at larger step sizes, but provides almost identical results as step size approaches 0. Simpson's rule can be shown as:

$$\int_a^b f(x)dx \approx \frac{h}{3}[f(x_0)] + 4 \sum_{i=1,3,5}^{N-1} f(x_i) + 2 \sum_{i=2,4,6}^{N-2} f(x_i) + f(x_N), h = \frac{b-a}{n}, x_i = a + \frac{2i-1}{2}h$$

Simpson's rule approximates the area under the curve by defining two endpoints, and two points in between. The function applies weights to each of the values, with higher weights assigned to the points in the middle. This provides a more accurate depiction of the function at higher step sizes. Despite this, at very low values of N, the method acts similarly to both the trapezoid and midpoint rule. As N increases, however the methods differ slightly in accuracy. The final method utilized, the 2-point Gauss-Quadrature Rule can be represented as:

$$\int_a^b f(x)dx \approx \frac{h}{2} \sum_{i=1}^N [f(x_i) + f(x_r)], x_l = \frac{1}{2}(x_i + x_{i-1} - \frac{h}{\sqrt{3}}), x_r = \frac{1}{2}(x_i + x_{i-1} + \frac{h}{\sqrt{3}}), h = \frac{b-a}{n}, x_i = a + \frac{2i-1}{2}h$$

The 2-point Gauss-Quadrature rule approximates integrals by choosing two non-arbitrary points along the line and assigning weights to each value. The selection and weights are defined by assuming that the function is exact for all polynomials less than third order. This assumption allows for the solving of all unknowns in the equation. This system allows for the method to be significantly more accurate than the preceding methods in this case.

3.2 Part II

For this problem, the code begins by storing the measured widths in an array. The number of subintervals is determined from the number of data points, and the spacing h is set equal to 1 foot. The program then separates the entries into odd and even indices so that the appropriate coefficients in Simpson's Rule can be applied. After summing the terms, the area is computed using the standard Simpson formula. The Simpson calculation produces an approximation of the total cross-sectional area of the land, it is an optimal method because it is able to accurately follow the curvatures of

the shape. It is then that the volume of mulch needed can be found, along with the bags needed.

3.3 Part III

Part 3 of the project uses Euler's Method on the differential equation with the given data provided above to get a plot of the simulation. Euler's Method works by replacing the actual curve with an approximation of tangent lines over small steps. Euler's Method is advantageous because it is simple and accurate for our needs. It is disadvantageous because it is only accurate in the first order. This can lead to inaccuracy over a period of time. Other methods may include Improved Euler's Method and Implicit Method. Euler's Method is preferable in this instance because it is boundary of what is covered in MA305.

4 Solutions/Results

4.1 Part I

Each method was implemented in a for loop and evaluated at different values of N , where N is the number of points considered in the interval. So as N increases, step size, h , decreases. The following figures show this implementation in Python. note that variables b and a remain constant, and h is based on N .

```
N = int(N_values[i]) #Define initial variables for ALL Rules
b = int(1)
a = float(0)
h = (b-a)/N
```

Figure 2: Defines variables a , b , and h depending on N

Values of N were arbitrarily defined between 0 and 10,000 to provide a reasonable showing of each methods approximation.

```

#Numerical integration - Midpoint Rule
x_mid = np.zeros(N+1) #Midpoint specific variable

for i in range(N+1): #Calculates midpoint of N sections - Produces sum and appends to (sum_mid)
    xi = a + ((2*i - 1)/2)*h
    func_mid = math.e**(-xi**2)
    x_mid[i] = func_mid
xsum_mid = h*(np.sum(x_mid))
pd_mid = ((abs(sum_actual-xsum_mid))/sum_actual)*100
sum_mid.append(xsum_mid)

```

Figure 3: Midpoint Rule implementation - For loop that appends to a larger array to store outputs from all values of N

```

#Numerical Integration - Trapezoid Rule
x_trap = np.zeros(N) #Trapezoid specific variables
x0 = a + h
xN = a + N*h

for i in range(N): #Calculates trapezoid rule for N sections - Produces sum and appends to (sum_trap)
    xi = a + i*h #Trapezoid specific variables
    func_trap = math.e**(-xi**2)
    x_trap[i] = func_trap

xsum_trap = h*((1/2)*math.e**(-x0**2) + np.sum(x_trap) + (1/2)*math.e**(-xN**2)) #Calculates trapezoidal sum - appends to (sum_trap)
pd_trap = ((abs(sum_actual-xsum_trap))/sum_actual)*100
sum_trap.append(xsum_trap)

```

Figure 4: Trapezoid Rule Implementation - note similarities to Midpoint Rule

```

#Numerical Integration - Simpson's Rule
x_odd = np.zeros(N) #Simpson's Rule specific variables
x_even = np.zeros(N-1)
x0 = a + h
xN = a + N*h

for i in range(0,N,2): #Calculates odd portion of Simpson's Rule
    xi = a + i*h
    func_odd = math.e**(-xi**2)
    x_odd[i] = func_odd

for i in range(1,N-1,2): #Calculates even portion of Simpson's Rule
    xi = a + i*h
    func_even = math.e**(-xi**2)
    x_even[i] = func_even

#Calculates sum using Simpson's Rule - appends to (sum_simp)
xsum_Simp = (h/3)*(math.e**(-x0**2) + 4*np.sum(x_odd) + 2*np.sum(x_even) + math.e**(-xN**2))
pd_Simp = ((abs(sum_actual-xsum_Simp))/sum_actual)*100
sum_simp.append(xsum_Simp)

```

Figure 5: Simpson's Rule Implementation - Note the even/odd steps denoted by two different variables (x_even,x_odd)

```

#Numerical Integration - 2-Point Gauss quadrature rule
x_gauss = np.zeros(N+1) #Gauss specific variable

for i in range(1,N+1): #Calculates sum with Gauss Rule - appends to (sum_gauss)
    xi = a + i*h
    xmin = a + (i-1)*h
    xl = (1/2) * (xi + xmin - (h/math.sqrt(3)))
    xr = (1/2) * (xi + xmin + (h/math.sqrt(3)))
    func_gauss = math.e**(-xl**2) + math.e**(-xr**2)
    x_gauss[i-1] = func_gauss

xsum_gauss = (h/2)*np.sum(x_gauss)
pd_gauss = ((abs(sum_actual-xsum_gauss))/sum_actual)*100
sum_gauss.append(xsum_gauss)

```

Figure 6: 2-point Gauss Quadrature Rule implementation - Algorithm is very simple considering its accuracy

The figures above give a much better idea of the actual use cases for these methods. This data is compiled at the end of each for loop and packed into arrays. These are then presented in tabular form through the use of Pandas Dataframes. Dataframes provide an easy way to display data in tabular format without having to iteratively print. This provides advantages both in speed and organization. As the for loops for each method run at slightly different times, data printed in a more traditional method may seem disorganized when displayed in the command window. The output is as shown in the figure below.

Real value: x= 0.746824132812427

	N	Midpoint Sum	Midpoint %Diff	Trapezoid Rule	Trapezoid %Diff	Simpsons Rule	Simpson %Diff	Gauss quadrature rule	Gauss %Diff
0	20	0.796870	6.701097	0.796608	6.666132	0.776826	4.017269	0.746824	2.850193e-07
1	100	0.756827	1.339380	0.756818	1.338115	0.753083	0.838124	0.746824	4.560717e-10
2	200	0.751925	0.669690	0.751923	0.669288	0.749969	0.421144	0.746824	2.849797e-11
3	500	0.748824	0.267817	0.748824	0.267767	0.748806	0.168953	0.746824	7.284302e-13
4	1000	0.747824	0.133904	0.747824	0.133892	0.747456	0.084559	0.746824	2.973185e-14
5	2000	0.747324	0.066951	0.747324	0.066948	0.747140	0.042300	0.746824	0.000000e+00
6	5000	0.747024	0.026780	0.747024	0.026780	0.746951	0.016925	0.746824	1.486592e-14
7	10000	0.746924	0.013390	0.746924	0.013390	0.746887	0.008463	0.746824	1.486592e-14

Figure 7: Output for part 1

The raw results (x-values) compared against increasing values of N can be shown in table 2.

N	Midpoint	Trapezoid	Simpson	Gauss Quadrature
20	0.796870	0.796608	0.776826	0.746824
100	0.756827	0.756818	0.753083	0.746824
200	0.751825	0.751823	0.749969	0.746824
500	0.748824	0.748824	0.748086	0.746824
1000	0.747824	0.747824	0.747456	0.746824
2000	0.747324	0.747324	0.747140	0.746824
5000	0.747024	0.747024	0.746951	0.746824
10000	0.746924	0.746924	0.746887	0.746824

Table 2: Raw output data of approximation with different methods

The data can be more easily visualized in a plot. Figure 8 displays the value of the integral determined by each method plotted against N. Viewing the data this way can easily show the diminishing returns for certain methods as N increases, as well as the requirements for very high values of N to reach a reasonable approximation for others.

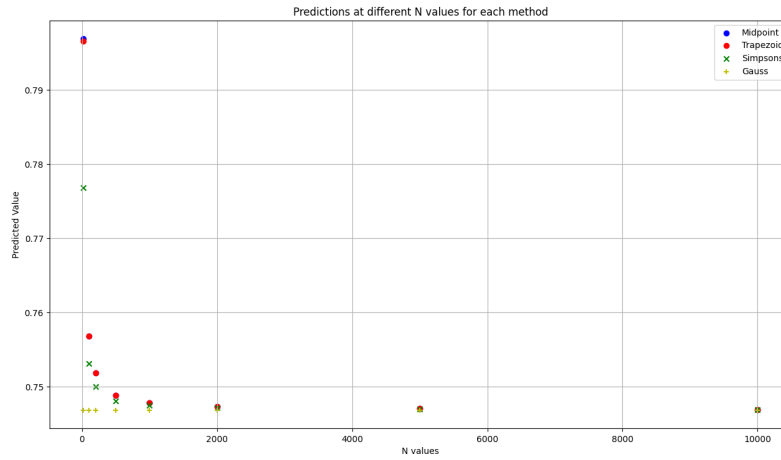


Figure 8: Plot of integral approximations vs. subdivides (N)

4.2 Part II

Using Simpson's Rule, the computed area of the land is:

$$A = 359.82716049382714 \text{ ft}^2.$$

The mulch depth is 0.25 feet, so the volume needed is:

$$V = A \times 0.25 = 89.95679012345678 \text{ ft}^3.$$

Each bag contains 3 cubic feet, so the number of bags is:

$$\text{bags} = \left\lceil \frac{89.95679012345678}{3} \right\rceil = \lceil 29.985596707818928 \rceil = 30.$$

These results show that the numerical method produces a reasonable estimate of the land area and the required mulch volume.

4.3 Part III

Using Euler's Method and a step size of $h = 1$ from 1950 to 2020 with the initial population at p_0 , the values corresponding to the time data are found. The plot below compares the Measured Population (in the millions) to the Year. It shows the given data points in comparison to Euler's Method. The general trend shows population growing upward with a discrepancy in the later years. This trend is continual with both plots. The Euler Method is more accurate as it gives a trend line versus just data points.

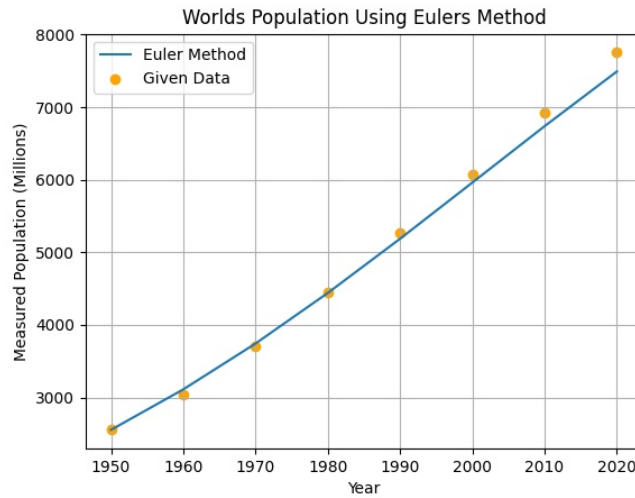


Figure 9: World population simulation using Euler's Method alongside given data.

5 Discussion/Conclusions

5.1 Part I

Part I of the project produced results that demonstrated the accuracy of each method and detailed the differences. The results show that across all values of N, the 2-Point Gauss-Quadrature rule was the most accurate. The error for each of the other methods decreases exponentially with the increase of N, and begin to approach the accuracy of the first method at very high values. The benefits provided by the Gauss-Quadrature rule make it stand out as an obvious choice for this kind of integration, assuming a low-order polynomial function. As functions become more complex, the advantages of this method will begin to fall away, likely providing similar results to what would be expected of the other methods. It can be shown that both the Midpoint Rule and the Trapezoid Rule provide answers that are very similar if not exactly the same at multiple of the tested intervals. This comes from the nature of the midpoint rule; as the endpoints move closer together, the average is closer to both values, providing an approximation that is much closer to the trapezoidal solution. As this portion of the project was purely theoretical, there is no physical interpretation of the results. With this being said, percent difference is the easiest metric to provide an understanding of the accuracy of the methods. Percent difference, defined as:

$$\%diff = \frac{|x_{theor} - x_{exp}|}{x_{theor}}$$

Allows scientists to gauge the precision of experimental answers as compared to theoretical data. It is used throughout the world to demonstrate improvements in methods and gauge the discrepancies between theoretical calculations and physical test methods. Shown below in table 3 is the percent difference data for each method against increasing values N.

N	Midpoint	Simpson's	Trapezoid	Gauss
100	1.339	0.8381	1.33815	4.561×10^{-10}
500	0.2678	0.1690	0.2678	7.284×10^{-13}
1000	0.1139	0.08456	0.1339	2.973×10^{-14}
2000	0.06695	0.04230	0.06695	0
5000	0.02678	0.01693	0.02678	1.4866×10^{-14}
10000	0.01339	0.008463	0.01339	1.4866×10^{-14}

Table 3: Table of percent differences for each method with respect to exact answer

This table provides an excellent showing of the accuracy of each method and demonstrates the much higher accuracy of the Gaussian-Quadrature Method. In conclusion, all methods provide a decent approximation for a required value, but each falls short in comparison to the Gaussian-Quadrature method. With respect to this, it should be stated that it is most likely that more than one method would be applied to a situation in order to ensure that both are providing reasonable results.

5.2 Part II

This part demonstrates how numerical methods such as Simpson's Rule can be used to approximate the area of an irregularly shaped region when direct geometric formulas are not applicable. The code effectively processes uniformly spaced measurements, applies the numerical integration method, and converts the computed area into practical quantities such as mulch volume and bags required.

The final results show that the land area is approximately 359.83 square feet, requiring about 89.96 cubic feet of mulch. To ensure adequate coverage, 30 bags of mulch must be purchased. The approach is efficient, and accurate for the given data.

Simpson's Rule is particularly well suited for this problem because the width measurements are evenly spaced and the underlying shape of the land is smooth rather than sharply changing. Compared to simpler numerical methods such as midpoint Riemann sums, Simpson's Rule provides significantly higher accuracy while using the same number of data points. It is more accurate than the trapezoidal rule would be because it uses quadratic approximations rather than straight-line segments. This makes Simpson's Rule ideal for estimating areas of natural, curved boundaries like the irregular region in this project.

The numerical method, together with the code implementation, produced a reliable estimate of the land area and the required volume of mulch. The method and implementation together offer an efficient and dependable way to solve real-world measurement problems.

5.3 Part III

For Part 3, Euler's Method shows that the population trend provides growth from 1950 to 2020. However, the first-order nature of Euler's Method gives inaccuracy the longer the time period. As far as the scope of this class, Euler's Method is adequate.

A Python Codes

Code referencing each part is provided.

```
1 #!/usr/bin/env python3
2
3
4 #Purpose: Implement Numerical methods to calculate a definite
      integral
5 #Methods: Midpoint rule, Trapezoid Rule, Simpson's Rule, 2-point
      Gauss quadrature rule
6 #Names: Wylie Roberts, Clancy Crawford, Hillary Spang
7 #Date: 10/08/25
8
9 import numpy as np
10 import math
11 import pandas as pd
12 import matplotlib.pyplot as plt
13
14 #Define initial variables
15 sum_actual = (1/2)*math.sqrt(math.pi)*math.erf(1)
16 N_values = [20, 100, 200, 500, 1000, 2000, 5000, 10000]
17
18 #Initiate Empty lists for iteration
19 values_table = []
20 sum_mid = []
21 sum_trap = []
22 sum_simp = []
23 sum_gauss = []
24
25 print("Real value: x=", sum_actual) #Print target sum
26 print()
27
28 #For loop to calculate sums at different values of N
29 for i in range(len(N_values)):
30
31     N = int(N_values[i]) #Define initial variables for ALL Rules
      - dependent on N so must be calculated each iteration
32     b = int(1)
33     a = float(0)
34     h = (b-a)/N
35
36     #Numerical integration - Midpoint Rule
37     x_mid = np.zeros(N+1) #Midpoint specific variable
38
39     for i in range(N+1): #Calculates midpoint of N sections -
      Produces sum and appends to (sum_mid)
40         xi = a + ((2*i - 1)/2)*h
41         func_mid = math.e**(-xi**2)
```

```

42     x_mid[i] = func_mid
43     xsum_mid = h*(np.sum(x_mid))
44     pd_mid = ((abs(sum_actual-xsum_mid))/sum_actual)*100
45     sum_mid.append(xsum_mid)
46
47
48     #Numerical Integration – Trapezoid Rule
49     x_trap = np.zeros(N) #Trapeziod specific variables
50     x0 = a + h
51     xN = a + N*h
52
53
54     for i in range(N): #Calculates trapezoid rule for N
55                             #sections – Produces sum and appends to (sum_trap)
56         xi = a + i*h #Trapezoid specific variables
57         func_trap = math.e**(-xi**2)
58         x_trap[i] = func_trap
59
60     xsum_trap = h*((1/2)*math.e**(-x0**2) + np.sum(x_trap) +
61 (1/2)*math.e**(-xN**2)) #Calculates trapezoidal sum – appends
62     to (sum_trap)
63     pd_trap = ((abs(sum_actual-xsum_trap))/sum_actual)*100
64     sum_trap.append(xsum_trap)
65
66
67     #Numerical Integration – Simpson's Rule
68
69     x_odd = np.zeros(N) #Simpson's Rule specific variables
70     x_even = np.zeros(N-1)
71     x0 = a + h
72     xN = a + N*h
73
74     for i in range(0,N,2): #Calculates odd portion of Simpson's
75                             #Rule
76         xi = a + i*h
77         func_odd = math.e**(-xi**2)
78         x_odd[i] = func_odd
79
80     for i in range(1,N-1,2): #Caluclates even portion of
81                             #Simpson's Rule
82         xi = a + i*h
83         func_even = math.e**(-xi**2)
84         x_even[i] = func_even
85
86     #Calulates sum using Simpson's Rule – appends to (sum_simp)
87     xsum_Simp = (h/3)*(math.e**(-x0**2) + 4*np.sum(x_odd) + 2*np
88 .sum(x_even) + math.e**(-xN**2))

```

```

85     pd_Simp = ((abs(sum_actual-xsum_Simp))/sum_actual)*100
86     sum_simp.append(xsum_Simp)
87
88     #Numerical Integration - 2-Point Gauss quadrature rule
89     x_gauss = np.zeros(N+1) #Gauss specific variable
90
91
92     for i in range(1,N+1): #Calculates sum with Gauss Rule -
93         #appends to (sum_gauss)
94         xi = a +i*h
95         ximin = a + (i-1)*h
96         xl = (1/2) * (xi + ximin - (h/math.sqrt(3)))
97         xr = (1/2) * (xi + ximin + (h/math.sqrt(3)))
98         func_gauss = math.e**(-xl**2) + math.e**(-xr**2)
99         x_gauss[i-1] = func_gauss
100
101     xsum_gauss = (h/2)*np.sum(x_gauss)
102     pd_gauss = ((abs(sum_actual-xsum_gauss))/sum_actual)*100
103     sum_gauss.append(xsum_gauss)
104
105     #Create a list of lists for tabulating data
106     values_table.append([N, xsum_mid, pd_mid, xsum_trap, pd_trap,
107         xsum_Simp, pd_Simp, xsum_gauss, pd_gauss])
108
109     #Convert to Dataframe with Pandas - Allows for easy tabulating
110     #without running in for loops
111     #All sums are appended to (values_table) and columns are labeled
112     #accordingly - it is then printed
113     df_list = pd.DataFrame(values_table, columns=['N', 'Midpoint Sum',
114         'Midpoint %Diff', 'Trapezoid Rule', 'Trapezoid %Diff',
115         'Simpson's Rule', 'Simpson %Diff', 'Gauss quadrature rule',
116         'Gauss %Diff'])
117     print(df_list)
118
119
120     #Plot values against expected sum
121     fig, ax = plt.subplots()
122     ax.scatter(N_values, sum_mid, color='b', label='Midpoint', marker=
123         'o')
124     ax.scatter(N_values, sum_trap, color='r', label='Trapezoid',
125         marker='o')
126     ax.scatter(N_values, sum_simp, color='g', label='Simpson's',
127         marker='x')
128     ax.scatter(N_values, sum_gauss, color='y', label='Gauss', marker=
129         '+')
130     ax.set_xlabel('N values')
131     ax.set_ylabel('Predicted Value')
132     ax.set_title('Predictions at different N values for each method')

```

```

122 ax.legend()
123 plt.grid(True)
124 plt.show()

1 #!/usr/bin/env python3
2 import math
3
4 """
5
6 MA305 – cw #: Part 2 – Hillary Spang – 11/17/25
7 Group: Wylie Roberts and Clancy Crawford
8 Purpose: Use Simpson's rule to approximate area of an
9           irregularly shaped area – extrapolate to volume with h=3 in.
10          #From this volume data – calculate how many bags of mulch (3 ft
11           ^3 each) are needed to fill volume
12
13 """
14 #import packages
15 import numpy as np
16 import matplotlib.pyplot as plt
17 import math
18
19 #Given input data
20 x = np.linspace(0,26,27)
21 y = np.array
22     ([8,10,11,15,16,16,16,16,15.5,15.5,15.5,15,15,15,14.5,14.5,
23      14,14,14,13.5,13,13,13,12,11,19,6])
24
25 #Simpson's Rule
26 N = 27 #Define initial variables
27 a = 0
28 b = 26
29 h = (b-a)/N
30 y_odd = np.zeros(N)
31 y_even = np.zeros(N)
32
33 #iterate for odd half of simpson's rule
34 for i in range(0,N,2):
35     xi = x[i]
36     func_odd = y[i]
37     y_odd[i] = func_odd
38
39 #iterate for even half
40 for i in range(1,N-1,2):
41     xi = x[i]
42     func_even = y[i]
43     y_even[i] = func_even

```

```

40
41 #Bags of mulch calculation from Simpson's Rule
42 area = (h/3)*(y[0] + 4*np.sum(y_odd) + 2*np.sum(y_even) + y[N
    -1]) #Formula for area from Simpson's Approx.
43 volume = area * (3/12) #3 inch layer of mulch (in ft)
44 bags = math.ceil(volume/3) #Number of bags of mulch needed to
    complete task
45
46 #print results
47 print('Area of land: ', area, 'ft^2')
48 print('Volume of mulch: ', volume, 'ft^3')
49 print('Bags needed: ', bags)

1 #!/usr/bin/env python3
2 #Purpose: Euler's Method to predict world population from
    1950–2020
3 #By: Clancy Crawford
4 #Group with: Wylie Roberts (Part 1) and Hillary Spang (Part 2)
5 #Date: 10/25/2025
6
7 import numpy as np
8 import math
9 import matplotlib.pyplot as plt
10
11
12 # Given Parameters
13 k_rm = 0.026 # per/yr
14 p_max = 12000 # million people
15 p0 = 2555 # million people in year start
16 yr_start = 1950
17 yr_end = 2020
18
19
20 # Given Function
21 def f(p):
22     return k_rm*(1 - (p/p_max))*p
23
24 # Given Data for time
25 t_data = np.arange(yr_start, yr_end + 1, 1) #must go up by one
    then extract later
26 t_len = len(t_data)
27 #print(t_data)
28
29 # Data for p
30 p_data_given = np.array([2555, 3040, 3708, 4454, 5276, 6079,
    6922, 7753], float)
31 p_data = np.zeros(t_len)
32 p_data[0] = p0
33

```

```

34 for i in range(t_len - 1):
35     p_data[i+1] = p_data[i] + 1 * f(p_data[i])
36
37
38 #Extracting Values
39 t_data_actual = np.arange(yr_start, yr_end + 1, 10)
40 index = t_data_actual - yr_start
41 p_data_actual = p_data[index]
42
43 print()
44 print("Years (t):", t_data_actual)
45 print("Population from Euler Method: ", p_data_actual)
46
47
48 # Plotting
49 plt.figure()
50 plt.plot(t_data_actual, p_data_actual, label="Euler Method")
51 plt.scatter(t_data_actual, p_data_given, color = 'orange', label
52             = 'Given Data')
53 plt.xlabel('Year')
54 plt.ylabel('Measured Population (Millions)')
55 plt.title('World's Population Using Euler's Method')
56 plt.grid(True)
57 plt.legend()
58 plt.savefig('project_part3_plot.jpg')
59 plt.show()

```